Finding an Error in a ChatGPT Conversation

Case study: PyTorch layer shapes and parameter counting

Student	**Mayer Hedda**
Neptun code	**DRWZQC**
Chatbot tested	ChatGPT (chatgpt.com)
Topic	Machine learning / deep learning (PyTorch CNNs)
Date	28 October 2026

2 errors found ChatGPT confidently computed the output of a PyTorch layer configuration that PyTorch rejects with a `ValueError`, and it made an arithmetic error when computing the output size of a transposed convolution. Both errors are confirmed by running the code.

1. Goal and method

The goal was to find a factual or reasoning error in a ChatGPT answer within the field of computer science / machine learning, and to document it in a verifiable way. I chose a task whose correct answer can be checked objectively by running code: computing the output shape and the number of trainable parameters of a small convolutional neural network written in PyTorch.

Method:

1. A first, simpler control question (Conv2d + BatchNorm + dilated Conv2d + MaxPool + Linear) was answered **correctly** by ChatGPT (3136 Linear inputs, 52,362 parameters). This shows that the model handles standard cases well.
2. A second question was designed with less common details: `padding='same'` combined with `stride=2`, grouped convolution (`groups=4`) and a transposed convolution with `output_padding`.
3. ChatGPT's answer was checked step by step by hand, then verified by running the model in PyTorch 2.15.

The prompt sent to ChatGPT

> Without running code, compute the total number of trainable parameters and the output shape for this PyTorch model with input (1, 16, 32, 32): Conv2d(16, 32, kernel_size=3, stride=2, padding='same', groups=4, bias=False) → BatchNorm2d(32) → ConvTranspose2d(32, 8, kernel_size=4, stride=2, padding=1, output_padding=1) → AdaptiveAvgPool2d(5) → Flatten → Linear(?, 10).

ChatGPT's final answer (summary)

Item	ChatGPT's claim	Reality (verified)
Layer 1 output (Conv2d, padding='same', stride=2)	(1, 32, 16, 16)	**Layer cannot be created – ValueError**
Layer 3 output (ConvTranspose2d)	(1, 8, 32, 32)	**(1, 8, 33, 33)**
Linear input size	200	**200 (correct, see section 5)**
Total trainable parameters	7,330	**7,330 (only if padding='same' is replaced)**
Final output shape	(1, 10)	**(1, 10) (same condition)**

2. Error 1 – a model that cannot exist

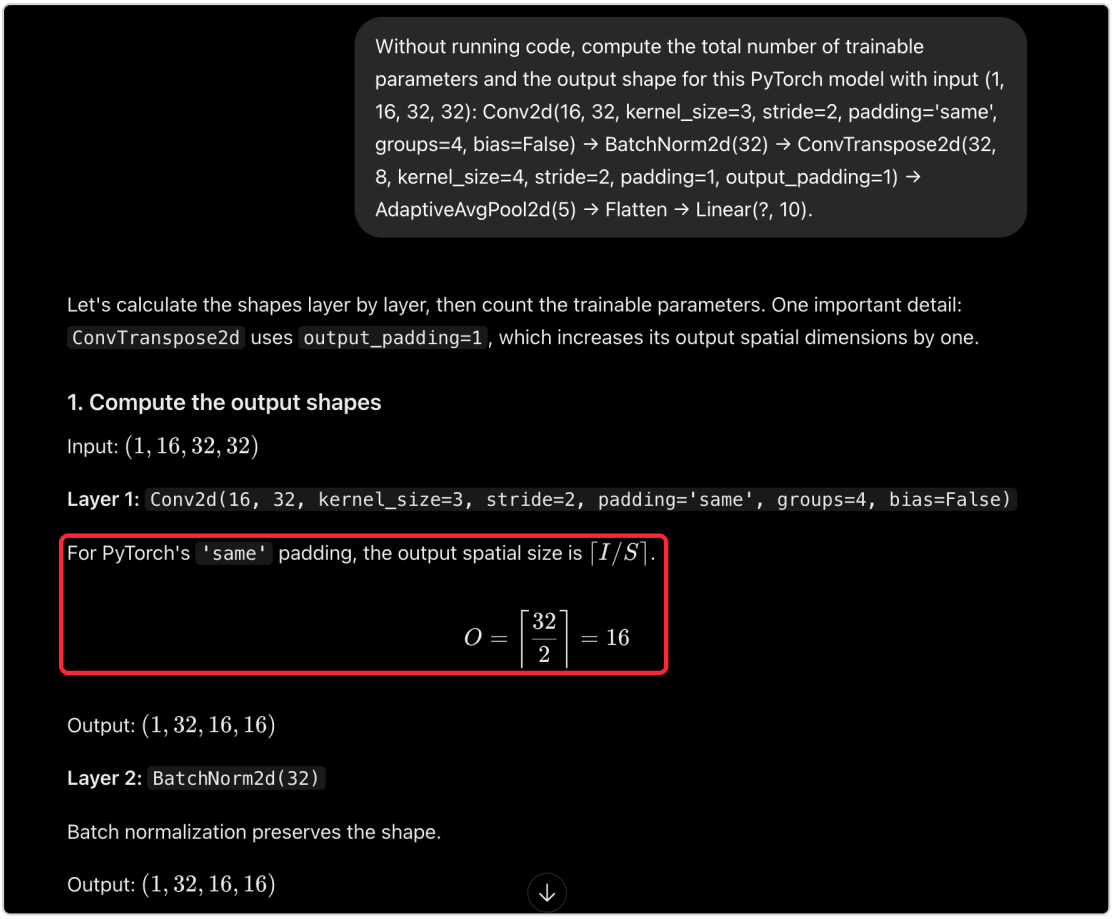


Figure 1 – The prompt and the first part of ChatGPT's answer. The red box marks the incorrect claim.

ChatGPT's claim: "For PyTorch's `'same'` padding, the output spatial size is ⌈I/S⌉", therefore the output is ⌈32/2⌉ = 16.

Reality: PyTorch does not support `padding='same'` for strided convolutions. The layer cannot even be constructed; `torch.nn.Conv2d` raises:
`ValueError: padding='same' is not supported for strided convolutions`

Why this is an error: In PyTorch, `padding='same'` is only allowed when `stride=1`. The ⌈I/S⌉ rule that ChatGPT used belongs to **TensorFlow/Keras**, where `padding="same"` with a stride is valid. ChatGPT mixed the behaviour of two different frameworks and presented it as a PyTorch fact. It then built the entire rest of the calculation on a model that would crash immediately when instantiated.

Error type: factual hallucination / false premise accepted. The model did not warn the user that the given code is invalid, which is the most important piece of information for anyone who would actually use this model.

3. Error 2 – arithmetic error in the transposed convolution

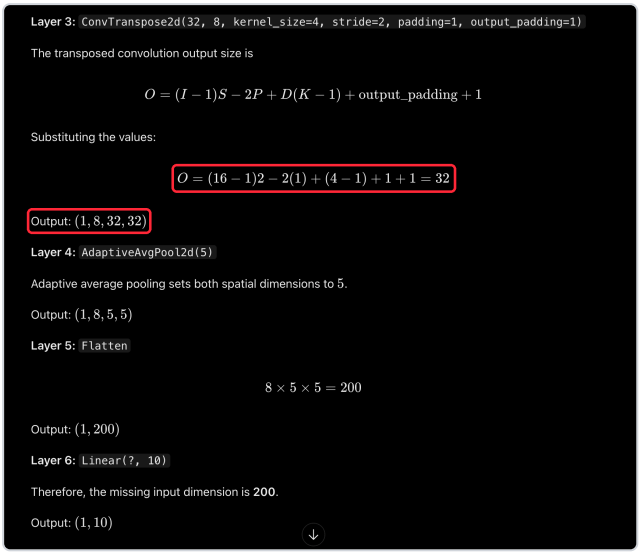


Figure 2 – ChatGPT substitutes the values correctly but computes the wrong result.

The formula ChatGPT wrote down is correct (it matches the PyTorch documentation):

```
O = (I − 1)·S − 2P + D·(K − 1) + output_padding + 1
```

Evaluating ChatGPT's own substitution step by step:

Term	Value
(I − 1)·S = (16 − 1)·2	30
− 2P = − 2·1	−2
D·(K − 1) = 1·(4 − 1)	+3
output_padding	+1
+ 1	+1
Total	**33 (ChatGPT wrote 32)**

> **ChatGPT:** output shape (1, 8, 32, 32)

> **Correct:** output shape (1, 8, 33, 33) – confirmed by PyTorch (section 4).

This error is especially notable because ChatGPT itself emphasised at the start of the answer that *output_padding=1 "increases its output spatial dimensions by one"*. Without output_padding the result would be exactly 32; so the final number silently ignores the very term the model highlighted. The written reasoning and the result contradict each other.

Error type: calculation error / inconsistency between reasoning steps and the result.

4. Verification by running the code

The following script was run with **PyTorch 2.15.0**. Test 3 uses the closest valid version of the model (`padding=1` instead of `'same'`), which is what produces a 16×16 output in PyTorch.

```
# verify.py
import torch, torch.nn as nn
print("PyTorch version:", torch.__version__)

# Test 1: the layer exactly as written in the prompt
try:
    nn.Conv2d(16, 32, kernel_size=3, stride=2, padding='same', groups=4, bias=False)
except Exception as e:
    print(f"Conv2d(padding='same', stride=2) -> {type(e).__name__}: {e}")

# Test 2: ConvTranspose2d output size (input 1x32x16x16)
ct = nn.ConvTranspose2d(32, 8, kernel_size=4, stride=2, padding=1, output_padding=1)
print("ConvTranspose2d output shape:", tuple(ct(torch.zeros(1, 32, 16, 16)).shape))

# Test 3: closest runnable model (padding=1 instead of 'same')
model = nn.Sequential(
    nn.Conv2d(16, 32, 3, stride=2, padding=1, groups=4, bias=False),
    nn.BatchNorm2d(32), ct, nn.AdaptiveAvgPool2d(5), nn.Flatten(), nn.Linear(200, 10))
print("Final output shape:", tuple(model(torch.zeros(1, 16, 32, 32)).shape))
print("Trainable parameters:", sum(p.numel() for p in model.parameters() if
p.requires_grad))
```

Output

```
PyTorch version: 2.15.0+cu132
Conv2d(padding='same', stride=2) -> ValueError: padding='same' is not supported for strided
convolutions
ConvTranspose2d output shape: (1, 8, 33, 33)
Final output shape: (1, 10)
Trainable parameters: 7330
```

The output confirms both errors: the original layer raises a `ValueError`, and the transposed convolution produces 33×33, not 32×32.

5. What ChatGPT got right – and why the errors are hidden

The parameter counting part of the answer is correct:

Layer	Calculation	Parameters
Conv2d (groups=4, no bias)	$32 \cdot (16/4) \cdot 3 \cdot 3$	1,152
BatchNorm2d(32)	$32\ \gamma + 32\ \beta$	64
ConvTranspose2d	$32 \cdot 8 \cdot 4 \cdot 4 + 8$	4,104
Linear(200, 10)	$200 \cdot 10 + 10$	2,010
Total		**7,330**

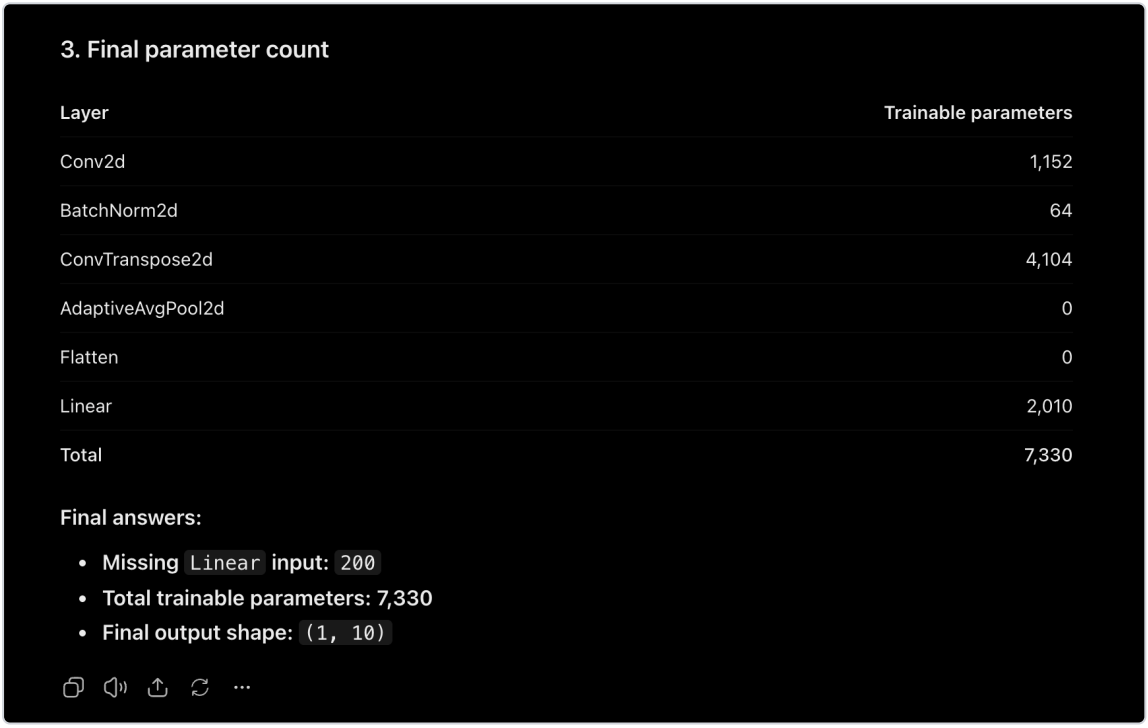


Figure 3 – ChatGPT's final summary. The numbers look correct, which hides the errors in the intermediate steps.

> **Why the final numbers still look right:** `AdaptiveAvgPool2d(5)` always outputs a 5×5 feature map, whatever the input size. It therefore "absorbs" the 33 vs. 32 mistake, and the Linear layer still receives 8·5·5 = 200 inputs. Parameter counts do not depend on the spatial size at all. A reader who only checks the final answer would conclude that ChatGPT was completely correct.

6. Conclusion

- **Error 1 (serious):** ChatGPT treated an invalid PyTorch configuration (`padding='same'` with `stride=2`) as valid, applying a TensorFlow/Keras rule to PyTorch. The model would crash on construction, but ChatGPT gave no warning.
- **Error 2 (calculation):** in the ConvTranspose2d step the correct formula and substitution led to a wrong result (32 instead of 33), contradicting ChatGPT's own remark about output_padding.
- **Hidden errors:** because of adaptive pooling, the final answers (200, 7,330, (1, 10)) look correct, so the mistakes are only visible by checking each intermediate step.

Lessons learned

- A correct-looking final answer does not guarantee correct reasoning. Intermediate steps must be checked as well.
- LLMs may mix up similar APIs of different libraries (here PyTorch vs. TensorFlow) and state framework-specific details with high confidence.
- The simple control question was answered correctly; errors appeared with less common parameter combinations. Edge cases are where LLM output most needs verification.
- For code-related questions, the most reliable check is to run the code: a few lines of PyTorch revealed both errors immediately.

Sources

- PyTorch documentation – `torch.nn.Conv2d` (padding='same' is not supported for strided convolutions): https://pytorch.org/docs/stable/generated/torch.nn.Conv2d.html
- PyTorch documentation – `torch.nn.ConvTranspose2d` (output size formula): https://pytorch.org/docs/stable/generated/torch.nn.ConvTranspose2d.html
- ChatGPT conversation, 28 October 2026 (screenshots in Figures 1–3).

Mayer Hedda · Neptun: DRWZQC